

ورشة GIT وGITHUB

التهيئة والتعريف بـ Git وGitHub

Session 01 • Setup & Orientation

- موقع ويب حقيقي على الإنترنت اسمه «دليل فريق الصف»
- كل طالب يضيف بطاقة تعريفية له بنفسه
- نرى البطاقات تتراكم واحداً بعد آخر حتى يصبح الموقع لوحة كاملة للفريق
- النهاية: رابط عام يفتحه أي شخص ويرى فيه اسمك ودورك

البرنامج كله مبني على مشروع واحد: تتعلم Git لأنك تحتاجه فعلاً، لا لأن هناك واجباً.

Git وGitHub ما الفرق بين

Git is a tool on your computer · GitHub is a service on the internet

Git — برنامج على جهازك

- يعمل بدون إنترنت وبدون حساب
- يتتبع التعديلات ويسجل الالتزامات
- ينشئ الفروع ويعيد الدمج ويحلّ التعارضات
- .git يحفظ التاريخ كاملاً في مجلد

local tool · no account · no internet needed

GitHub — خدمة على الإنترنت

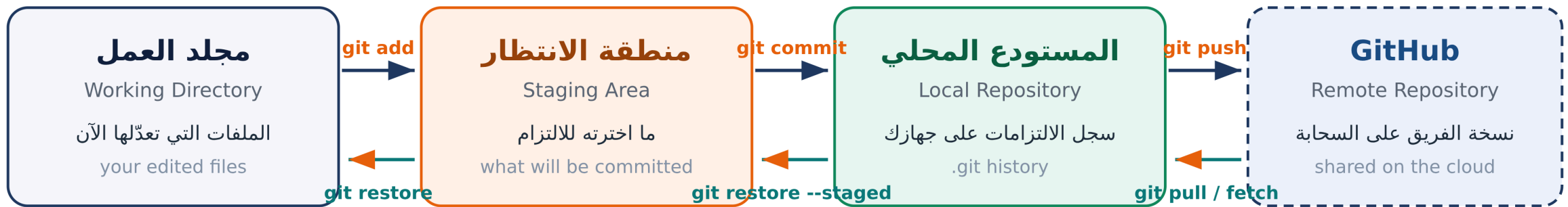
- يحتاج حساباً واتصالاً بالإنترنت
- يستضيف المستودعات ويعمل نسخاً احتياطياً
- طلبات السحب والمراجعة والتعليقات
- المشكلات، الإجراءات، ونشر الصفحات

online service · account required · teamwork

هو المكان الذي نجتمع فيه — ولا يعمل أحدهما بدل الآخر GitHub هو المحرّك، و Git: العلاقة

- Git هو المحرّك: برنامج على جهازك يسجّل التغييرات ويدير الفروع
- GitHub هو المكان: خدمة على الإنترنت يستضيف المستودعات ويضيف التعاون
- لا يعمل أحدهما بدل الآخر: بدون Git لا تعمل من الطرفية، وبدون GitHub لا يعمل الفريق معاً
- سؤال سريع: الأمر `git status` أين يعمل؟ — على جهازك

اختبر نفسك: أي الأمرين يحتاج إنترنت؟ `git commit` أم `git push`؟ (الجواب: الثاني فقط)



القاعدة: لكل التزام رسالة واضحة، ولكل تعديل كبير فرع خاص.

Rule: every commit needs a clear message; every big change gets its own branch.

الأمر	المنطقة	التشبيه
git status	لا يعدّل — يعرض الحالة	لوحة القيادة
git add	إلى منطقة الانتظار	تحضير الطلب
git commit	إلى المستودع المحلي	تسليم الطلب
git push	إلى GitHub	إرسال للفرع
git pull	من GitHub إلى جهازك	استلام تحديثات الزملاء

الترتيب الإلزامي: push ← commit ← add. لا يمكن تخطي خطوة.

- المهمة 1: تثبيت Git والتحقق منه
- المهمة 2: ضبط الهوية (الاسم والبريد) — أهم خطوة في الورقة
- المهمة 3: إنشاء حساب GitHub مهني + التحقق بخطوتين
- المهمة 4: مفتاح SSH أو رمز وصول شخصي (PAT)
- المهمة 5: تجهيز VS Code والطرفية
- المهمة 6: أول مستودع شخصي: clone ثم commit ثم push

القاعدة الذهبية: اكتب الأوامر بيدك، ولا تخف من الخطأ — كل خطأ هنا يمكن التراجع عنه.

```
$ git config --global user.name "سارة الحسن"  
$ git config --global user.email "sara@example.com"  
$ git config --global init.defaultBranch main  
$ git config --global pull.rebase true
```

استخدم نفس البريد المسجل في حساب GitHub. إن اختلف البريد، لن يرتبط التزامك بحسابك وقد يظهر باسم غريب بلا صورة.

```
$ cd ~/projects
$ git clone https://github.com/<user>/git-practice-<user>.git
$ cd git-practice-<user>

# ثم عدّل README.md:
$ git status          # ماذا تغيّر؟
$ git add README.md
$ git commit -m "docs(readme): add a short personal introduction"
$ git push
$ git log --oneline
3f1a9c2 (HEAD → main, origin/main) docs(readme): add a short personal introduction
9d4b1a7 Initial commit
```

ما معنى «دليل» في هذه الورشة؟

- لا نقيّم ما تقوله، بل ما تُنتجه: مخرجات أوامر الحقيقية من جهازك
- كل مهمة فيها صندوق «دليل التسليم» تلصق فيه المخرجات
- الأدلة المنسوخة من الورقة تُرفض — نعرفها من رقم الإصدار وأسماء الملفات
- إتقان الأمر أهم من سرعة إنجازه

معيّار إتمام اليوم: حساب مؤمّن بـ Git + 2FA مهيّأ بالبريد الصحيح + مستودع شخصي فيه التزامك الأول.

- أكملت المهام 1-6 وملأت الأدلة في ورقة العمل 01
- تأكد: `git config --list` يُظهر بريدك الصحيح
- تأكد: `git log --oneline` في مستودعك يُظهر التزامك
- الجلسة القادمة: سنستنسخ **مستودع الفريق** ونشغل الموقع على أجهزتنا

سؤال الليلة: لماذا لا يكفي `git add` وحدها لرفع عملك إلى GitHub؟ ستجد الجواب في أول دقيقة من الجلسة القادمة.